# MEMORIA PROYECTO REDES II (2023-24)



Fuente: Marenostrum 5 - Xataka


## Integrantes “Grupo 1”:

- Germán Pajarero (german.pajarero@alu.uclm.es)
- Houda El Ouahabi (houda.elouahabi@alu.uclm.es)
- Oumaima Darkaoui (oumaima.darkaoui@alu.uclm.es)
- Zineb El Ouaazizi (zineb.elouaazizi@alu.uclm.es)

# ÍNDICE:

**Estructura del Proyecto:** **3**
**Creación y Configuración de Instancias AWS:** **4**
Creación de Servidor: 4
Configuración de Servidor: 4
Creación de Cliente: 5
**Configuración de Cliente:** **5**
**¿Cómo ejecutar el Proyecto?:** **6**
Servidor: 6
Cliente: 6
Petición LIST: 6
Petición SEND: 7
Petición RECEIVE: 7
Petición EXEC: 7
Algunas Capturas de la Ejecución: 8
**Análisis de Wireshark:** **9**
Modo Normal: 9
Modo SSL: 9
**Descripción de Funcionamiento del Código Fuente:** **10**
Clase RCS: 10
Clase RCC: 11
Clase ServerThread: 12
**Funcionalidad Extra:** **13**
Script para Envío a Través de SCP (Batch y Bash): 13
Script para Conexión Automática a Terminales SSH de Instancias AWS (Bash): 13
Ejecución en Bucle: 13
La Petición “LIST” Muestra Archivos y Carpetas: 13
Creación Automática de Archivos y Carpetas de Prueba: 13
JavaDoc Web: 14

**JAVADOC**

# Estructura del Proyecto:

- **Directorio “certs”:**
    - Contiene los archivos relacionados con el uso de SSL (cacerts, certificados, ...)
- **Directorio “logs”:**
    - En él se almacenan los logs resultantes de la ejecución del programa, tanto para servidor como para cliente.
- **Directorio “server_files”:**
    - Contiene los archivos que se encontrarán dentro del servidor y con los que probaremos los comandos.
- **Directorio “client_files” (o como decida llamarlo el usuario, el nombre es parte de los argumentos):**
    - Contiene los archivos que se encontrarán dentro del cliente y con los que probaremos los comandos.
- **Directorio “src”:**
    - Contiene el código fuente de nuestro proyecto, está dividido en tres partes:
        - **RCS:** Corresponde al código fuente del servidor.
        - **ServerThread:** Corresponde al código fuente del del hilo que creará el servidor para cada cliente.
        - **RCC:** Corresponde al código fuente del cliente.
- **Directorio “extras”:**
    - Contiene archivos que hemos desarrollado como funcionalidades extras del proyecto.
- **Directorio “wireshark”:**
    - Contiene las capturas de wireshark.

# Creación y Configuración de Instancias AWS:

## Creación de Servidor:

Creamos una instancia con las siguientes características:

- **Ubuntu Server**
- **T2.micro**
- **Nuevas Keys.**
- **Permitir Tráfico SSH.**

Permitimos el tráfico de entrada en esa instancia por el puerto **8008** (TCP).

## Configuración de Servidor:

```
sudo apt update && sudo apt upgrade
```

```
sudo apt install openjdk-22-jdk
```

```
mkdir proyectoRedes
```

```
mkdir proyectoRedes/certs
```

```
cd proyectoRedes/certs
```

```
keytool -genkey -keyalg RSA -alias serverKey -keystore serverKey.jks -storepass
servpass
```

```
keytool -export -keystore serverKey.jks -alias serverKey -file
ServerPublicKey.cer
```

Con esto se instala lo necesario y se crea el certificado de servidor, mediante el cual obtenemos con -export su clave pública, que después añadiremos a cacerts en el cliente.

# Creación de Cliente:

Creamos una instancia con las siguientes características:

- **Ubuntu Server**
- **T2.micro**
- **Nuevas Keys.**
- **Permitir Tráfico SSH.**

Permitimos el tráfico de entrada en esa instancia por el puerto **8008** (TCP).

# Configuración de Cliente:

```
sudo apt update && sudo apt upgrade
```

```
sudo apt install openjdk-22-jdk
```

```
mkdir proyectoRedes
```

```
mkdir proyectoRedes/certs
```

Ahora debemos enviar la clave pública del servidor al cliente, nosotros lo hacemos desde nuestro PC para no tener que enviar keys SSH a las instancias:

```
scp -i <ruta ssh key> <origen> <destino>
```

- Origen es la ruta remota de la clave pública del servidor.
- Destino es una ubicación de nuestro PC.

```
scp -i <ruta ssh key> <origen> <destino>
```

- Origen es la ruta de nuestro PC en la que hemos guardado la clave pública del servidor.
- Destino es la ruta remota del cliente en la que guardaremos la clave pública del servidor.

```
keytool -import -alias serverKey -file ServerPublicKey.cer -keystore cacerts
-keypass changeit -storepass changeit
```

Con esto importamos la clave pública del servidor en el cacerts del cliente.

# ¿Cómo ejecutar el Proyecto?:

## Servidor:

- **java RCS <modo> <puerto> <máximo número de clientes>**
    - **Modo:** Puede ser “normal” o “ssl” (en minúsculas)
    - **Puerto:** Nosotros hemos escogido el 8008.
    - **Máximo número de clientes:** La cantidad que se quiera especificar, si es menor que 1 muestra error.

## Cliente:

- **java RCC <modo> <host> <puerto> <carpeta cliente>**
    - **Modo:** Puede ser “normal” o “ssl” (en minúsculas).
    - **Host:** IP del servidor.
    - **Puerto:** Nosotros hemos escogido el 8008.
    - **Carpeta de cliente:** Nombre que queremos para la carpeta en la que se almacenarán los archivos pertenecientes al cliente.
        - El código incluye un script para rellenarla con elementos de prueba.

**IMPORTANTE: “server_files” y “client_folder” (o como se decida llamar), SE CONSIDERAN RAÍZ AL USAR LOS COMANDOS.**

- **server_files** = /
- **client_folder** = /

## Petición LIST:

- **LIST <directorio>**
    - Si lo que queremos es mostrar lo que hay dentro de la raíz (consideramos server_files como raiz), escribimos:
        - LIST /
    - Si lo que queremos es mostrar otro directorio que haya dentro, por ejemplo:
        - LIST /dir1

# Petición SEND:

- **SEND <archivo local> <destino remoto>**
    - Ejemplo:
        - SEND /test.txt / (Se envía el archivo test.txt de la raíz del cliente a la raíz del servidor)

# Petición RECEIVE:

- **RECEIVE <archivo remoto> <destino local>**
    - Ejemplo:
        - RECEIVE /test.txt / (Se recibe el archivo test.txt del remoto y se guarda en la raíz de local).

# Petición EXEC:

- **EXEC <comando con sus argumentos>**
    - Ejemplo:
        - EXEC ip a (Se envía el comando y se devuelve la respuesta)

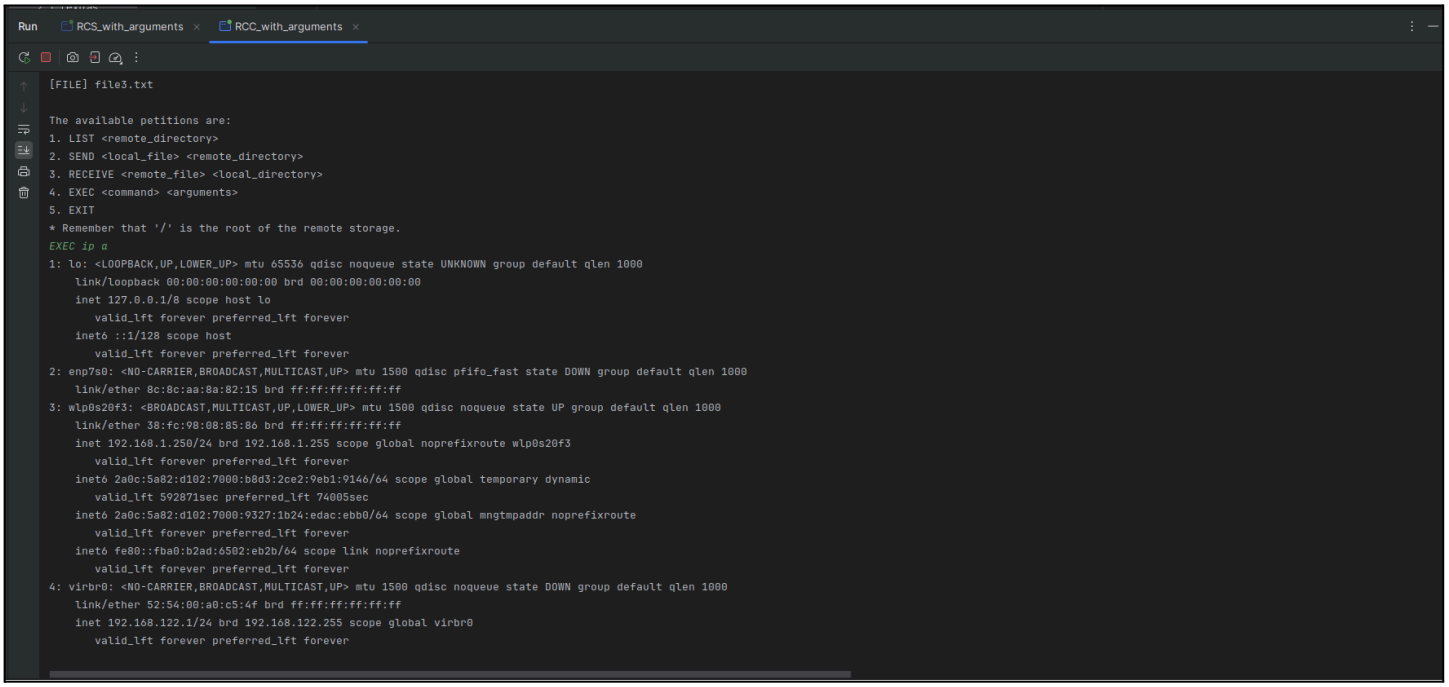
**Toda la ejecución mostrada en la demo se ha realizado en las instancias del usuario german.pajarero@alu.uclm.es**

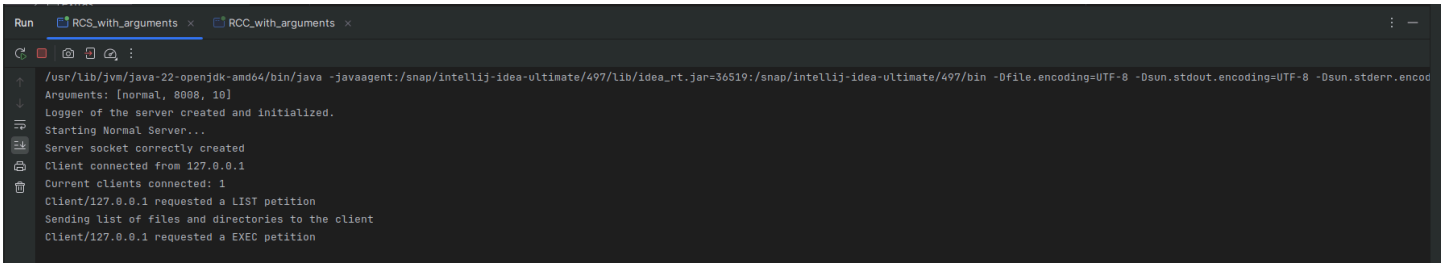
# Algunas Capturas de la Ejecución:





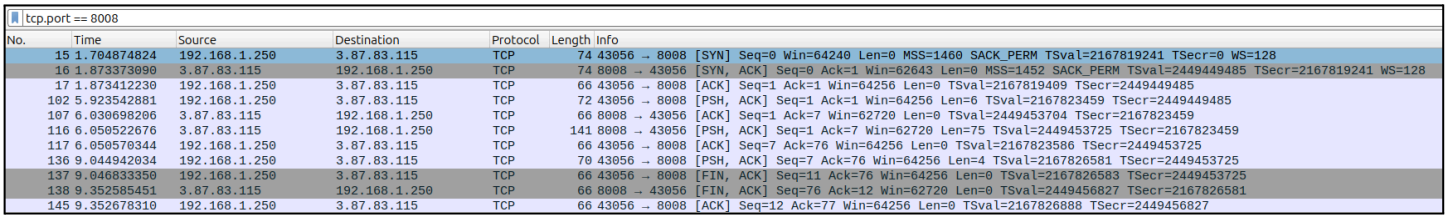




# Análisis de Wireshark:

## Modo Normal:



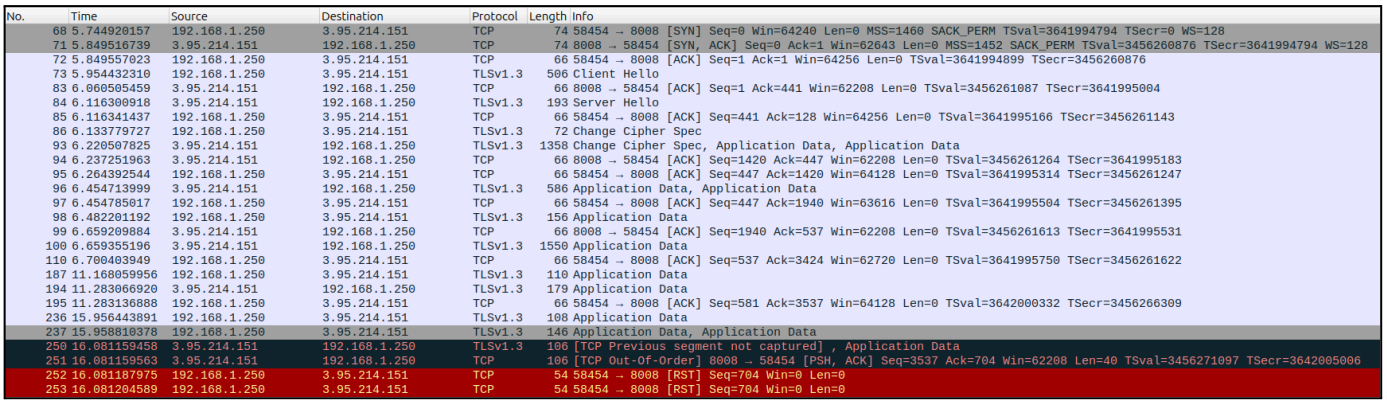
tcp.port == 8008

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 15 | 1.704874824 | 192.168.1.250 | 3.87.83.115 | TCP | 74 | 43056 → 8008 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2167819241 TSecr=0 WS=128 |
| 16 | 1.873373090 | 3.87.83.115 | 192.168.1.250 | TCP | 74 | 8008 → 43056 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=1452 SACK_PERM TSval=2449449485 TSecr=2167819241 WS=128 |
| 17 | 1.873412230 | 192.168.1.250 | 3.87.83.115 | TCP | 66 | 43056 → 8008 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2167819409 TSecr=2449449485 |
| 102 | 5.923542881 | 192.168.1.250 | 3.87.83.115 | TCP | 72 | 43056 → 8008 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=6 TSval=2167823459 TSecr=2449449485 |
| 107 | 6.030698206 | 3.87.83.115 | 192.168.1.250 | TCP | 66 | 8008 → 43056 [ACK] Seq=1 Ack=7 Win=62720 Len=0 TSval=2449453704 TSecr=2167823459 |
| 116 | 6.050522676 | 3.87.83.115 | 192.168.1.250 | TCP | 141 | 8008 → 43056 [PSH, ACK] Seq=1 Ack=7 Win=62720 Len=75 TSval=2449453725 TSecr=2167823459 |
| 117 | 6.050570344 | 192.168.1.250 | 3.87.83.115 | TCP | 66 | 43056 → 8008 [ACK] Seq=7 Ack=76 Win=64256 Len=0 TSval=2167823586 TSecr=2449453725 |
| 136 | 9.044942034 | 192.168.1.250 | 3.87.83.115 | TCP | 70 | 43056 → 8008 [PSH, ACK] Seq=7 Ack=76 Win=64256 Len=4 TSval=2167826581 TSecr=2449453725 |
| 137 | 9.046833350 | 192.168.1.250 | 3.87.83.115 | TCP | 66 | 43056 → 8008 [FIN, ACK] Seq=11 Ack=76 Win=64256 Len=0 TSval=2167826583 TSecr=2449453725 |
| 138 | 9.352585451 | 3.87.83.115 | 192.168.1.250 | TCP | 66 | 8008 → 43056 [FIN, ACK] Seq=76 Ack=12 Win=62720 Len=0 TSval=2449456827 TSecr=2167826581 |
| 145 | 9.352678310 | 192.168.1.250 | 3.87.83.115 | TCP | 66 | 43056 → 8008 [ACK] Seq=12 Ack=77 Win=64256 Len=0 TSval=2167826888 TSecr=2449456827 |

En la captura de WireShark podemos ver varios pasos que se producen en la comunicación.

1. **[Paq 15]** → El cliente envía al servidor un paquete con la flag SYN, informando de que desea empezar a transmitir mediante TCP.
2. **[Paq 16]** → El servidor responde con una flag ACK (recibido).
3. **[Paq 17]** → El cliente responde con otro ACK (recibido), se establece conexión TCP.
4. **[Paq 102]** → El cliente envía datos al servidor, flag PSH (push), examinando el paquete se puede ver en claro que en este caso es “LIST /”.
5. **[Paq 107]** → El servidor devuelve ACK.
6. **[Paq 116]** → En el siguiente PSH se puede ver la respuesta del servidor al cliente en texto claro.
7. **[Paq 117]** → ACK por parte del cliente.
8. **[Paq 136]** →Cliente envía un “EXIT”, flag PSH.
9. **[Paq 137]** → Cliente envía un “FIN” para finalizar conexión.
10. **[Paq 138]** →Servidor envía otro “FIN” y “ACK” del anterior paquete, se finaliza.
11. **[Paq 145]** → Cliente envía “ACK”, se puede dar por finalizada la conexión.

## Modo SSL:



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 68 | 5.744920157 | 192.168.1.250 | 3.95.214.151 | TCP | 74 | 58454 → 8008 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3641994794 TSecr=0 WS=128 |
| 71 | 5.849516739 | 3.95.214.151 | 192.168.1.250 | TCP | 74 | 8008 → 58454 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=1452 SACK_PERM TSval=3456260876 TSecr=3641994794 WS=128 |
| 72 | 5.849557023 | 192.168.1.250 | 3.95.214.151 | TCP | 66 | 58454 → 8008 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3641994899 TSecr=3456260876 |
| 73 | 5.954432310 | 192.168.1.250 | 3.95.214.151 | TLSv1.3 | 506 | Client Hello |
| 83 | 6.060505459 | 3.95.214.151 | 192.168.1.250 | TCP | 66 | 8008 → 58454 [ACK] Seq=1 Ack=441 Win=62208 Len=0 TSval=3456261087 TSecr=3641995004 |
| 84 | 6.116300918 | 3.95.214.151 | 192.168.1.250 | TLSv1.3 | 193 | Server Hello |
| 85 | 6.116341437 | 192.168.1.250 | 3.95.214.151 | TCP | 66 | 58454 → 8008 [ACK] Seq=441 Ack=128 Win=64256 Len=0 TSval=3641995166 TSecr=3456261143 |
| 86 | 6.133779727 | 192.168.1.250 | 3.95.214.151 | TLSv1.3 | 72 | Change Cipher Spec |
| 93 | 6.220507825 | 3.95.214.151 | 192.168.1.250 | TLSv1.3 | 1358 | Change Cipher Spec, Application Data, Application Data |
| 94 | 6.237251963 | 3.95.214.151 | 192.168.1.250 | TCP | 66 | 8008 → 58454 [ACK] Seq=1420 Ack=447 Win=62208 Len=0 TSval=3456261264 TSecr=3641995183 |
| 95 | 6.264392544 | 192.168.1.250 | 3.95.214.151 | TCP | 66 | 58454 → 8008 [ACK] Seq=447 Ack=1420 Win=64128 Len=0 TSval=3641995314 TSecr=3456261247 |
| 96 | 6.454713999 | 3.95.214.151 | 192.168.1.250 | TLSv1.3 | 586 | Application Data, Application Data |
| 97 | 6.454785017 | 192.168.1.250 | 3.95.214.151 | TCP | 66 | 58454 → 8008 [ACK] Seq=447 Ack=1940 Win=63616 Len=0 TSval=3641995504 TSecr=3456261395 |
| 98 | 6.482201192 | 192.168.1.250 | 3.95.214.151 | TLSv1.3 | 156 | Application Data |
| 99 | 6.659209884 | 3.95.214.151 | 192.168.1.250 | TCP | 66 | 8008 → 58454 [ACK] Seq=1940 Ack=537 Win=62208 Len=0 TSval=3456261613 TSecr=3641995531 |
| 100 | 6.659355196 | 3.95.214.151 | 192.168.1.250 | TLSv1.3 | 1550 | Application Data |
| 110 | 6.700403949 | 192.168.1.250 | 3.95.214.151 | TCP | 66 | 58454 → 8008 [ACK] Seq=537 Ack=3424 Win=62720 Len=0 TSval=3641995750 TSecr=3456261622 |
| 187 | 11.168059956 | 192.168.1.250 | 3.95.214.151 | TLSv1.3 | 110 | Application Data |
| 194 | 11.283066920 | 3.95.214.151 | 192.168.1.250 | TLSv1.3 | 179 | Application Data |
| 195 | 11.283136888 | 192.168.1.250 | 3.95.214.151 | TCP | 66 | 58454 → 8008 [ACK] Seq=581 Ack=3537 Win=64128 Len=0 TSval=3642000332 TSecr=3456266309 |
| 236 | 15.956443891 | 192.168.1.250 | 3.95.214.151 | TLSv1.3 | 108 | Application Data |
| 237 | 15.958810378 | 192.168.1.250 | 3.95.214.151 | TLSv1.3 | 146 | Application Data, Application Data |
| 250 | 16.081159458 | 3.95.214.151 | 192.168.1.250 | TLSv1.3 | 106 | [TCP Previous segment not captured] , Application Data |
| 251 | 16.081159563 | 3.95.214.151 | 192.168.1.250 | TCP | 106 | [TCP Out-Of-Order] 8008 → 58454 [PSH, ACK] Seq=3537 Ack=704 Win=62208 Len=40 TSval=3456271097 TSecr=3642005006 |
| 252 | 16.081187975 | 192.168.1.250 | 3.95.214.151 | TCP | 54 | 58454 → 8008 [RST] Seq=704 Win=0 Len=0 |
| 253 | 16.081204589 | 192.168.1.250 | 3.95.214.151 | TCP | 54 | 58454 → 8008 [RST] Seq=704 Win=0 Len=0 |

Analizado los diversos paquetes en Wireshark se puede ver como el contenido está ofuscado y no permite ver en claro las peticiones realizadas al servidor.

# Descripción de Funcionamiento del Código Fuente:

## Clase RCS:

Esta clase contiene el código perteneciente al servidor, contiene los siguientes métodos:

- **Main:**
  - Método principal de la clase, a través de él se llama a los métodos que realizan la ejecución en cadena.
- **CheckServerArgs:**
  - Método que se encarga de comprobar que los argumentos introducidos sean correctos.
- **StartServer:**
  - Método que inicia la ejecución del servidor, bifurca en función de si se desea modo normal o SSL.
- **RunNormalServer:**
  - Inicia el servidor en modo normal, crea el socket y queda a la espera de clientes en un bucle que impide que haya más clientes de los permitidos.
- **RunSSLServer:**
  - Inicia el servidor en modo SSL:
    - Carga el keystore del servidor.
    - Mediante un key manager se accede a las claves.
    - Genera el socket.
  - El método queda a la espera de clientes en un bucle que impide que haya más clientes de los permitidos.
- **StartLogger:**
  - Método que inicia la herramienta que nos permitirá registrar las acciones del servidor en un archivo .log en la carpeta /logs.
- **checkLogsFolder:**
  - Método que comprueba si la carpeta /logs existe, si no existe, la crea.
- **checkFilesDirectory:**
  - Comprueba que la carpeta de archivos del servidor exista, sino la crea y la llena de archivos/directorios de prueba.

# Clase RCC:

Esta clase se encarga de implementar la funcionalidad del cliente, contiene los siguientes métodos:

- **Main:**
    - Método principal de la clase, a través de él se llama a los métodos que realizan la ejecución en cadena.
- **CheckClientArgs:**
    - Método que se encarga de comprobar que los argumentos introducidos sean correctos.
- **StartClient:**
    - Método que inicia la ejecución del cliente, bifurca en función de si se desea modo normal o SSL.
- **RunNormalClient:**
    - Inicia el cliente en modo normal, crea el socket e intenta la conexión con la IP y puerto especificados.
- **RunSSLClient:**
    - Inicia el cliente en modo SSL:
        - Carga el trusted store (cacerts).
        - Inicializa contexto SSL.
        - Creación de socket factory y socket cliente.
        - Creación de evento para la detección de handshake.
        - Inicio del handshake.
- **RunPetitions:**
    - Gestiona la solicitud de peticiones, desviando mediante un switch a la petición seleccionada.
- **ListPetition:**
    - Comando LIST, envía la ruta requerida al servidor y éste devuelve los archivos y directorios que hay en la misma (especificando el tipo).
- **SendPetition:**
    - Comando SEND, envía un archivo local al servidor. Verifica la existencia del archivo local y el directorio remoto antes de realizar la transferencia.
- **ReceivePetition:**
    - Comando RECEIVE, recibe un archivo del servidor y lo guarda en el directoria local especificado. Verifica la existencia del directorio local antes de recibir el archivo.
- **ExecPetition:**
    - Comando EXEC, envía un comando para ejecutar en el servidor y recibe la respuesta del servidor.
- **ExitPetititon:**
    - Comando EXIT, finaliza la ejecución del cliente.
- **StartLogger:**
    - Método que inicia la herramienta que registra las acciones del cliente en un archivo .log en la carpeta /logs.

- **CheckLogsFolder:**
    - Método que comprueba si la carpeta /logs existe, si no existe, la crea.
- **CheckFilesDirectory:**
    - Comprueba que la carpeta de archivos del cliente exista, sino la crea y la llena de archivos/directorios de prueba.
- **ShowOptions:**
    - Método que imprime las opciones disponibles de ejecución en el cliente.

# Clase ServerThread:

Esta clase se encarga de implementar la funcionalidad de cada hilo de ejecución, contiene los siguientes métodos:

- **Run:**
    - Método principal de la clase, desde el se llama a un método de forma indefinida que procesa peticiones.
- **ProcessPetition:**
    - Método que se encarga de leer la petición que llega por el flujo de datos y seleccionar mediante un switch el camino correspondiente que seguirá en la ejecución.
- **ListPetition:**
    - Comando LIST, este método se encarga de listar los archivos y directorios en el servidor según la solicitud del cliente. Si el directorio no existe, se envía un mensaje de error al cliente. Si el directorio existe, se envía la lista de archivos y subdirectorios al cliente.
- **SendPetition:**
    - Comando SEND, este método envía un archivo desde el servidor al cliente. Primero, verifica si el archivo existe. Si el archivo no existe, se envía un mensaje de error al cliente. Si el archivo existe, se envía el tamaño del archivo y luego el contenido del archivo en sí.
- **ReceivePetititon:**
    - Comando RECEIVE, este método recibe un archivo desde el cliente y lo guarda en el servidor. Similar a SendPetition, primero verifica si el archivo existe y luego procede a enviar el tamaño del archivo y su contenido al cliente.
- **ExecPetition**:
    - Comando EXEC, este método ejecuta un comando del sistema operativo en el servidor a solicitud del cliente, retornando la respuesta del comando solicitado.
- **ExitPetition:**
    - Comando EXIT, este método cierra la conexión con el cliente. Se registra en el log que la conexión se ha cerrado, se decrementa el contador de clientes actuales, y se detiene el bucle infinito de procesamiento de peticiones.

# Funcionalidad Extra:

En este apartado describimos algunas de las funcionalidades extras que hemos implementado en el proyecto de forma voluntaria:

## Script para Envío a Través de SCP (Batch y Bash):

Hemos creado un script en batch y bash para hacer el envío de los códigos fuente a las instancias de AWS EC2 más rápido y ágil.
Está incluido en la carpeta “extras”.

## Script para Conexión Automática a Terminales SSH de Instancias AWS (Bash):

Hemos creado un script mediante el cual podemos conectarnos a ambas instancias en dos pestañas de terminal, haciendo el proceso de conexión mucho más ágil.
Está incluido en la carpeta “extras”.

* Hemos usado Batch y Bash en los scripts para mostrar que se puede hacer lo mismo en diferentes SO y con diferentes herramientas.

## Ejecución en Bucle:

El cliente se ejecuta en bucle hasta que el usuario introduce la petición “EXIT”.

## La Petición “LIST” Muestra Archivos y Carpetas:

La petición LIST muestra un tag al principio del nombre indicando si el elemento es un archivo o un directorio.

## Creación Automática de Archivos y Carpetas de Prueba:

Se ha implementado un método en el servidor y el cliente que sirve para crear un directorio que contiene una serie de ficheros y otros directorios con los que poder probar el funcionamiento del programa.

# JavaDoc Web:

JavaDoc alojado en una página web mediante la cual poder visualizarlo cómodamente.

Enlace al JavaDoc Web